

OpenFOAM: Casos, Código e Dicas

CFD: Curso Básico

Professor: Adriano Possebon Rosa

Departamento de Engenharia Mecânica
Universidade de Brasília

Sumário

- 1 Introdução
- 2 Escoamento em Degrau
- 3 OpenFOAM - Pastas
- 4 OpenFOAM - Tutoriais
 - damBreak
 - damBreakFine em Paralelo
 - pitzDailySteady
 - airFoil2D
 - coolingCylinder2D
- 5 OpenFOAM - Dicas

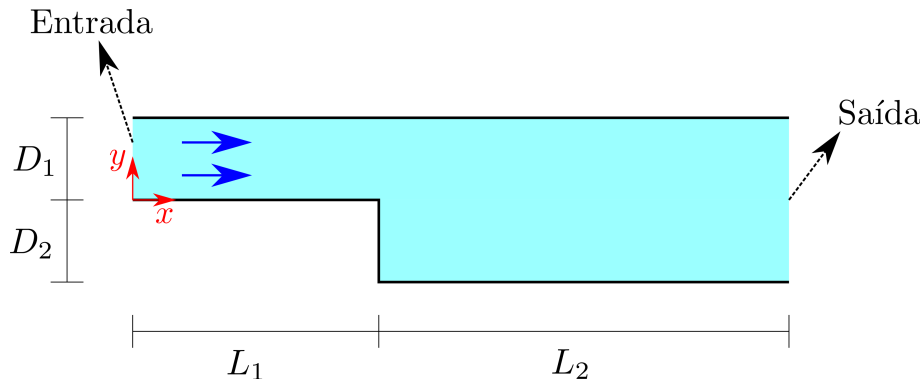
Nesta aula vamos começar com um estudo detalhado do problema do escoamento em um degrau.

Depois, vamos conhecer melhor a estrutura do OpenFOAM, vamos rodar alguns tutoriais e vamos ver algumas dicas.

Sumário

- 1 Introdução
- 2 Escoamento em Degrau
- 3 OpenFOAM - Pastas
- 4 OpenFOAM - Tutoriais
 - damBreak
 - damBreakFine em Paralelo
 - pitzDailySteady
 - airFoil2D
 - coolingCylinder2D
- 5 OpenFOAM - Dicas

Vamos resolver o problema do escoamento de um fluido newtoniano incompressível em um degrau, como mostra a figura.



O fluido entra pela esquerda, com uma velocidade uniforme U_{in} , e sai pela direita.

A entrada tem um comprimento L_1 e a saída tem um comprimento L_2 .

O canal tem uma largura inicial D_1 e depois a largura se torna $D_1 + D_2$.

O número de Reynolds desse escoamento é definido como

$$Re = \frac{U_{in} D_1}{\nu} ,$$

em que ν é a viscosidade cinemática do fluido.

Vamos usar o módulo *incompressibleFluid*.

Para começar, vamos copiar o caso do escoamento entre duas placas paralelas.

Temos as 3 pastas:

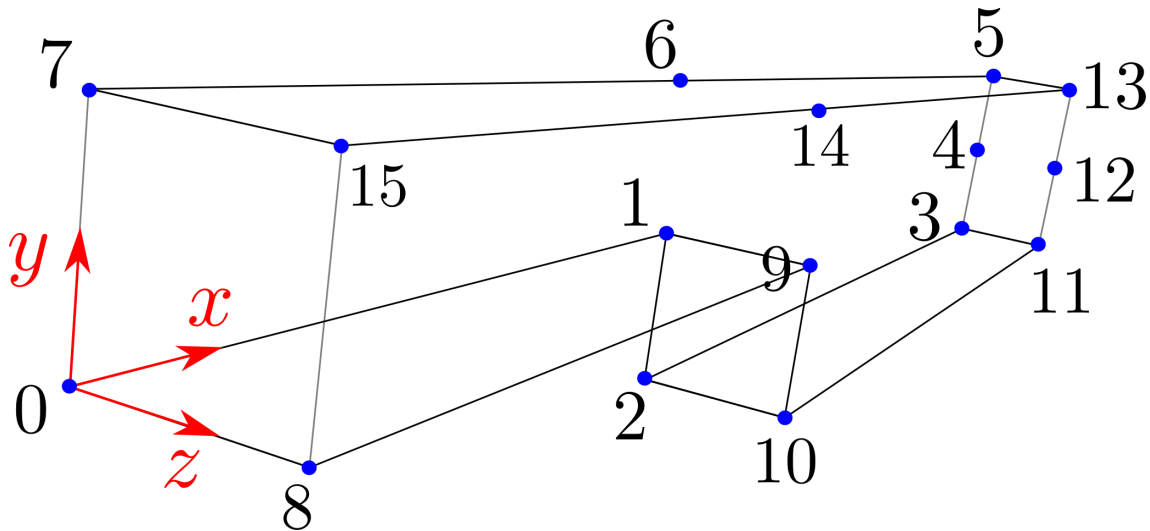
- 0
- constant
- system

Passos:

- gerar a geometria e a malha,
- configurar as propriedades físicas e as condições de contorno,
- configurar os parâmetros da simulação,
- rodar a simulação e
- visualizar o resultado.

Vamos começar pela geometria e malha.

A geometria do problema é apresentada na figura abaixo.



Temos 16 pontos.

Esses pontos são adicionados no arquivo *blockMeshDict*, na pasta *system*.

system/blockMeshDict

```
1  #include "../parameters"
2  convertToMeters 1.0;
3  vertices
4  (
5      (0 0 0)                // 0
6      ($L1 0 0)              // 1
7      ($L1 #calc "-$D2" 0)   // 2
8      (#calc "$L1 + $L2" #calc "-$D2" 0) // 3
9      (#calc "$L1 + $L2" 0 0) // 4
10     (#calc "$L1 + $L2" $D1 0) // 5
11     ($L1 $D1 0)             // 6
12     (0 $D1 0)               // 7
```

```
13 (0 0 $Lz) // 8
14 ($L1 0 $Lz) // 9
15 ($L1 #calc "-$D2" $Lz) // 10
16 (#calc "$L1 + $L2" #calc "-$D2" $Lz) // 11
17 (#calc "$L1 + $L2" 0 $Lz) // 12
18 (#calc "$L1 + $L2" $D1 $Lz) // 13
19 ($L1 $D1 $Lz) // 14
20 (0 $D1 $Lz) // 15
21 );
```

Todos os pontos estão na lista *vertices*.

Agora estamos usando variáveis nossas para representar a posição dos pontos. Isso facilita muito na hora de rodar casos diferentes.

As variáveis estão definidas no arquivo *parameters*, que está na pasta principal da simulação, junto com as pastas *0*, *constant* e *system*.

Este arquivo é apresentado no próximo slide.

parameters

```
1 // Geometry
2
3 D1 0.1;
4 D2 $D1;
5 L1 #calc "5*$D1";
6 L2 #calc "10*$D1";
7 Lz 0.1; // This is the length in z direction.
8 // This value is not important in this simulation.
9
10 // Mesh
11
12 Nx1 50; // Number of cells in the x direction, for the entrance.
13 Nx2 100;
14 Ny1 10;
15 Ny2 10;
```

Para criar uma nova variável, basta incluir, em uma nova linha, o nome da variável, um espaço em branco, o seu valor e terminar com ponto e vírgula.

Por exemplo, para a criar uma variável chamada $D1$ com valor 0.1, basta adicionar ao código

$$D1 \ 0.1;$$

Além disso, é possível fazer operações com as variáveis já definidas. Para isso podemos usar a função *#calc*.

Queremos que o comprimento de entrada *L1* seja 5 vezes a largura de entrada *D1*. Assim:

$$L1 \text{ \#calc "5 * \$D1" ;}$$

Note que é necessário colocar a operação entre aspas.

O OpenFOAM possui diversas funções implementadas, como seno, cosseno e potência. Depois veremos mais dessas funções.

Voltando ao *blockMeshDict*, dê uma olhada na lista *vertices* para entender a posição de cada ponto e para ver se está tudo certo.

Note que o arquivo *parameters* foi incluído no começo do arquivo *blockMeshDict* com o comando

```
#include "../parameters"
```

O próximo passo no `blockMeshDict` é incluir os blocos com as células. Nosso domínio está dividido em 3 blocos: um na entrada e dois na saída.

system/blockMeshDict

```
22 blocks
23 (
24     hex (0 1 6 7 8 9 14 15) ($Nx1 $Ny1 1) simpleGrading (1 1 1)
25     hex (1 4 5 6 9 12 13 14) ($Nx2 $Ny1 1) simpleGrading (1 1 1)
26     hex (2 3 4 1 10 11 12 9) ($Nx2 $Ny2 1) simpleGrading (1 1 1)
27 );
```

O número de divisões de cada bloco em cada uma das direções aparece na frente da definição do bloco.

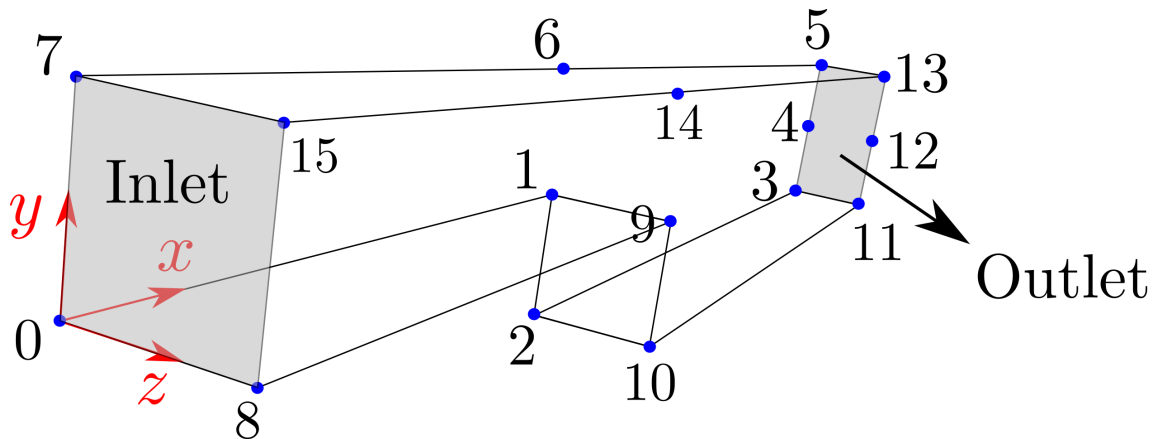
Neste caso, o primeiro bloco tem $Nx1$ divisões na direção x , $Ny1$ divisões na direção y e 1 célula somente na direção z .

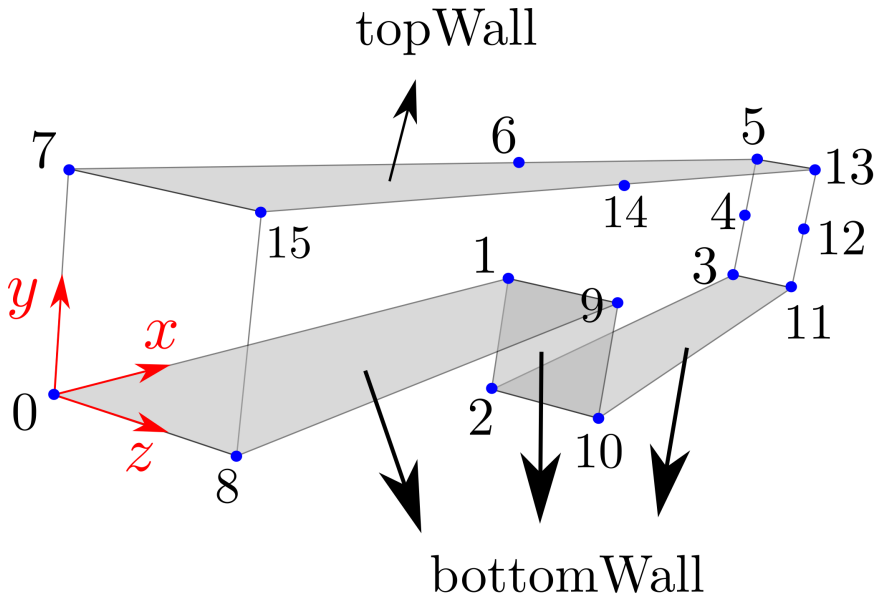
Comentário: para mudar a geometria e o número de pontos, basta alterar o arquivo *parameters*. Isso facilita muito a nossa vida.

Continuando no *blockMeshDict* vamos definir as superfícies de contorno. Isso é feito na lista *boundary*.

Não é necessário incluir nenhuma informação sobre superfícies internas.

Quando uma superfície tem mais de um retângulo, é necessário informar cada retângulo.





system/blockMeshDict

```
28 boundary
29 (
30     inlet
31     {
32         type patch;
33         faces
34         (
35             (0 8 15 7)
36         );
37     }
38     outlet
39     {
40         type patch;
41         faces
42         (
43             (3 11 12 4)
44             (4 12 13 5)
45         );
46     }
```

```
47 topWall
48 {
49     type wall;
50     faces
51     (
52         (15 14 6 7)
53         (14 13 5 6)
54     );
55 }
56 bottomWall
57 {
58     type wall;
59     faces
60     (
61         (8 9 1 0)
62         (2 10 9 1)
63         (10 11 3 2)
64     );
65 }
```



```
67
68     frontAndBack
69     {
70         type empty;
71         faces
72         (
73             (8 9 14 15)
74             (10 11 12 9)
75             (12 13 14 9)
76             (0 1 6 7)
77             (2 3 4 1)
78             (1 4 5 6)
79         );
80     }
81 );
```

Rode o comando

\$: blockMesh

e verifique se está tudo certo com a malha.

Na pasta *constant*, vamos informar que a simulação é laminar no arquivo *momentumTransport* e vamos informar a viscosidade cinemática do fluido no arquivo *physicalProperties*.

system/momentumTransport

```
1 simulationType laminar;
```

system/physicalProperties

```
2  #include "../parameters"
3
4  viscosityModel constant;
5
6  nu          [0 2 -1 0 0 0 0] $nu;
```

Note que o valor da viscosidade é definido a partir da variável ν (nu), que é foi criada pelo usuário (nós, no caso).

A variável ν (nu) é definida no arquivo *parameters*.

O parâmetro que governa esse escoamento é o número de *Reynolds*, definido como

$$Re = \frac{U_{in} D_1}{\nu} .$$

Com a largura da entrada D_1 definida e a velocidade de entrada U_{in} definida, o único parâmetro que podemos modificar é ν , para obter um determinado Re .

Assim, para um determinado Re , podemos definir a viscosidade como

$$\nu = \frac{U_{in} D_1}{Re} .$$

É essa conta que aparece no arquivo *parameters*, no próximo slide.

Comentário: nos experimentos, ν é o parâmetro mais difícil de ajustar, já que depende do fluido em questão. Mas nas simulações é bem tranquilo mudar o ν para obter um determinado Re .

parameters

```
16 // Inlet velocity
17
18 Uinlet 0.1;
19
20 // Reynolds
21 //  $Re = Uinlet * D1 / nu \rightarrow nu = Uinlet * D1 / Re$ 
22
23 Re 10;
24
25 nu #calc "$Uinlet * $D1 / $Re";
```

Agora vamos para a pasta 0 para impor as condições iniciais e de contorno.

$0/U$

```
1  #include "../parameters"
2
3  dimensions      [0 1 -1 0 0 0 0];
4
5  internalField uniform (0 0 0);
6
7  boundaryField
8  {
9      inlet
10     {
11         type          fixedValue;
12         value          uniform ($Uinlet 0 0);
13     }
14
15     outlet
16     {
17         type          zeroGradient;
18     }
```

```
19  topWall
20  {
21      type          fixedValue;
22      value          uniform (0 0 0);
23  }
24
25  bottomWall
26  {
27      type          fixedValue;
28      value          uniform (0 0 0);
29  }
30
31  frontAndBack
32  {
33      type          empty;
34  }
35 }
```

0/p

```
1 dimensions      [0 2 -2 0 0 0 0];
2
3 internalField uniform 0;
4
5 boundaryField
6 {
7     inlet
8     {
9         type      zeroGradient;
10    }
11
12    outlet
13    {
14        type      fixedValue;
15        value      uniform 0;
16    }
```

```
18
19     topWall
20     {
21         type            zeroGradient;
22     }
23
24     bottomWall
25     {
26         type            zeroGradient;
27     }
28
29     frontAndBack
30     {
31         type            empty;
32     }
33 }
```

Os ajustes da simulação são feitos no arquivo *controlDict*.

system/controlDict

```
1  #include "../parameters"
2
3  application    foamRun;
4  solver         incompressibleFluid;
5
6  startFrom      latestTime;
7
8  startTime      0;
9
10 stopAt         endTime;
11
12 endTime        $finalTime;
13
14 deltaT          $dt;
15
16 writeControl    adjustableRunTime;
17
18 writeInterval   $writeInterval;
19
20 purgeWrite      0;
```

```
22
23 writeFormat    ascii;
24
25 writePrecision 6;
26
27 writeCompression off;
28
29 timeFormat      general;
30
31 timePrecision 6;
32
33 runTimeModifiable true;
34
35
36 functions
37 {
38     #includeFunc    residuals;
39
40     #includeFunc    grad;
41
42     #includeFunc    patchFlowRate;
43 }
```

Note que usamos as variáveis *finalTime*, *dt* e *writeInterval*, que são definidas pelo usuário (sei disso pela presença do símbolo \$ no início do nome da variável).

Essas variáveis também estão definidas no arquivo *parameters*.

parameters

```
26 // Control
27
28 finalTime 5.0;
29
30 dt 0.01;
31
32 writeInterval 0.5;
```


Comentário: se você está usando o Windows, muito cuidado ao criar o arquivo *parameters*, pois o Windows pode não reconhecer esse arquivo como sendo um arquivo de texto.

Uma forma segura de criar o arquivo *parameters*, é copiar um arquivo de texto (*blockMeshdict*, por exemplo), apagar tudo que está neste arquivo, renomeá-lo como *parameters.txt* e aí sim escrever no arquivo.

Se você fizer isso, lembre-se de modificar a linha de *include* para

#include “../*parameters.txt*”

Note que no final do arquivo *controlDict* nós temos o dicionário *functions*. Esse dicionário permite incluir funções (*functionObjects*) que serão executadas junto com a simulação.

Com essas funções, você pode calcular o fluxo através de uma superfície, a quantidade de calor, o gradiente da velocidade, a vorticidade, etc.

Para ver uma lista completa de todas as funções disponíveis, rode o comando

```
$: foamToC -functionObjects
```

ou o comando

```
$: foamPostProcess -list
```

Dê uma olhada com calma nessa lista, você vai encontrar várias funções interessantes e que podem ser úteis na sua pesquisa.

Nós vamos usar 3 aqui na nossa simulação: *residuals*, *grad* e *patchFlowRate*.

A função *residuals* vai registrar o resíduo (erro) na solução das variáveis. Vamos ver mais sobre isso em outras aulas.

A função *grad* vai retornar o gradiente do vetor velocidade (um tensor). Vamos usar esse gradiente para determinar o tamanho da recirculação.

A função *patchFlowRate* nos dá a vazão de fluido através de uma superfície. Iremos utilizá-la para verificar se a vazão na saída é igual à da entrada.

É necessário importar um arquivo correspondente a cada função e colocar esse arquivo na pasta *system*.

Isso é feito automaticamente pelo comando *foamGet*. Rode os seguintes comandos:

```
$: foamGet residuals
```

```
$: foamGet grad
```

```
$: foamGet patchFlowRate
```

Verifique se esses arquivos apareceram na pasta *system*.

Configure esses arquivos de acordo com o apresentado nos próximos slides.

system/residuals

```
1 #includeEtc "caseDicts/postProcessing/numerical/residuals.cfg"  
2  
3 fields (p U);
```

system/grad

```
1 type          grad;  
2 libs          ("libfieldFunctionObjects.so");  
3  
4 field         U;  
5  
6 executeControl writeTime;  
7 writeControl  writeTime;
```

system/patchFlowRate

```
1 patch outlet;  
2  
3 fields (phi);  
4  
5 operation sum;  
6  
7 #includeEtc "caseDicts/postProcessing/surfaceFieldValue/patchValue.cfg"
```


Por fim, os arquivos *fvSchemes* e *fvSolution* são apresentados nos próximos slides.

Ainda não vamos entrar nos detalhes desses arquivos.

system/fvSchemes

```

1  ddtSchemes
2  {
3  default      Euler;
4  }
5
6  gradSchemes
7  {
8  default      Gauss linear;
9  grad(p)      Gauss linear;
10 }
11
12 divSchemes
13 {
14 default      none;
15 div(phi,U)   Gauss linear;
16 }

```

```

17 laplacianSchemes
18 {
19 default      Gauss linear orthogonal;
20 }
21
22 interpolationSchemes
23 {
24 default      linear;
25 }
26
27 snGradSchemes
28 {
29 default      orthogonal;
30 }

```

system/fvSolution

```

1  solvers
2  {
3  p
4  {
5  solver      PCG;
6  preconditioner DIC;
7  tolerance    1e-06;
8  relTol      0;
9  }
10
11 pFinal
12 {
13 $p;
14 relTol      0;
15 }

```

```

16
17 "(U|UFinal)"
18 {
19 solver      smoothSolver;
20 smoother    symGaussSeidel;
21 tolerance    1e-05;
22 relTol      0;
23 }
24 }
25
26 PIMPLE
27 {
28 nCorrectors    2;
29 nNonOrthogonalCorrectors 0;
30 pRefCell      0;
31 pRefValue     0;
32 }

```

Estamos prontos para rodar a simulação.

Execute o comando

```
$: foamRun
```

Você pode acompanhar o valor dos resíduos ao longo da simulação. Em uma outra janela, execute

```
$: foamMonitor -l postProcessing/residuals/0/residuals.dat
```

Pronto! A simulação rodou!!

Foram criadas pastas com os resultados para determinados passos de tempo.

Abra a primeira pasta depois da 0 . Agora nós temos um arquivo para a variável $\nabla \mathbf{U}$ ($\text{grad}(\mathbf{U})$).

Note que também foi criada uma pasta chamada *postProcessing*, com o resíduo e o fluxo através da saída.

Na pasta *postProcessing*, veja o valor da vazão volumétrica de fluido na saída. A massa se conserva?

Vamos visualizar o resultado.

Opção 1: execute os comandos

```
$: touch foam.foam
```

```
$: paraview foam.foam
```

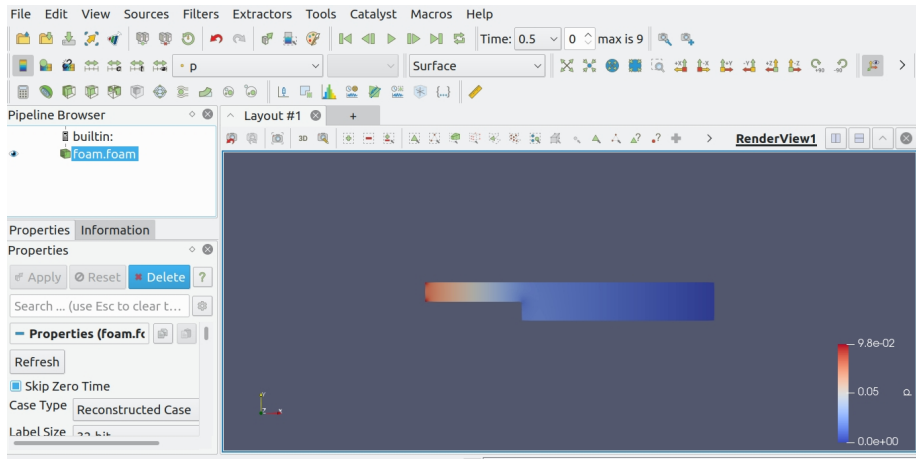
Opção 2: execute o comando

```
$: paraFoam
```

Opção 3: abra o paraview diretamente do Windows.

No paraview, clique em *Apply*, na esquerda ao centro.

Você vai ver esta tela:



Depois,

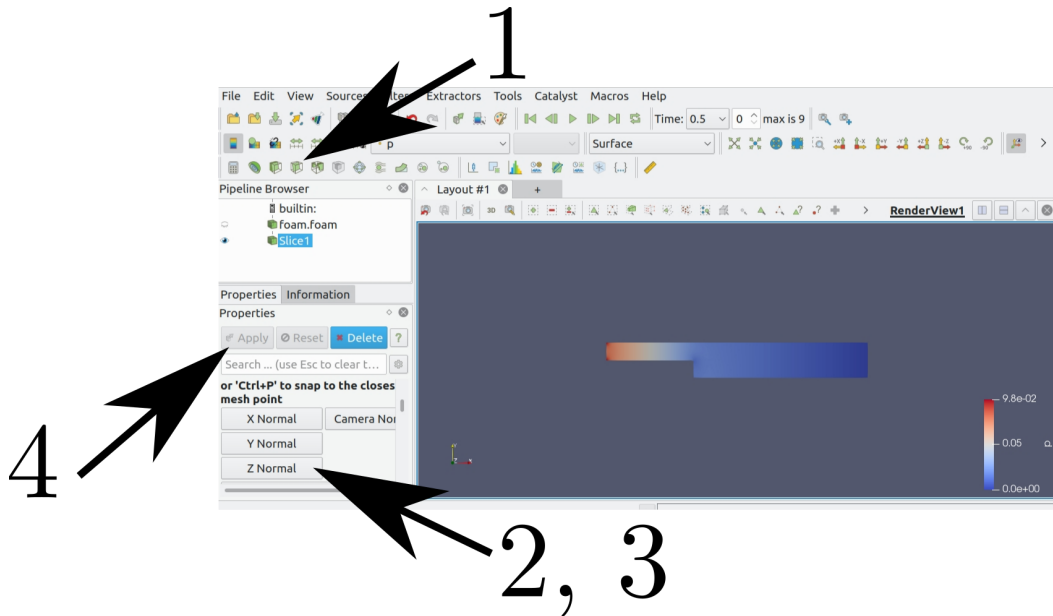
1 clique em *Slice*, no canto esquerdo superior.

2 em propriedades, no canto esquerdo central, clique em *Z Normal*

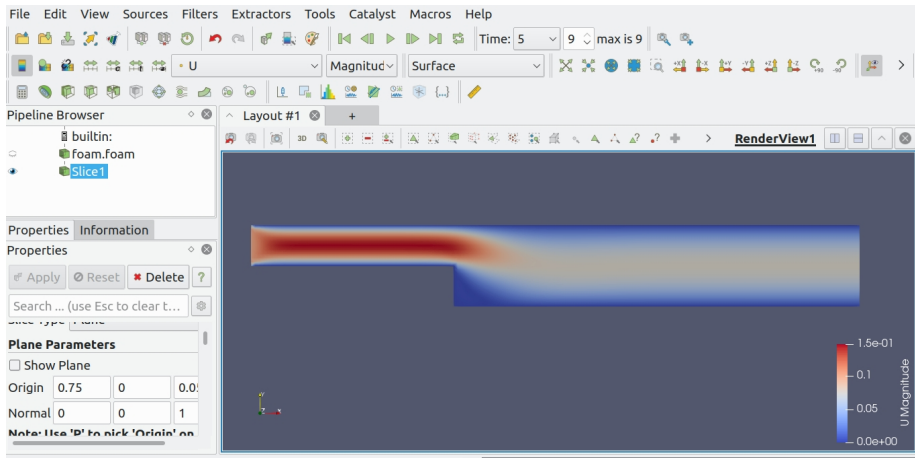
3 ainda em propriedades, desmarque a opção *Show Plane*

4 Clique em *Apply*

Veja o próximo slide.

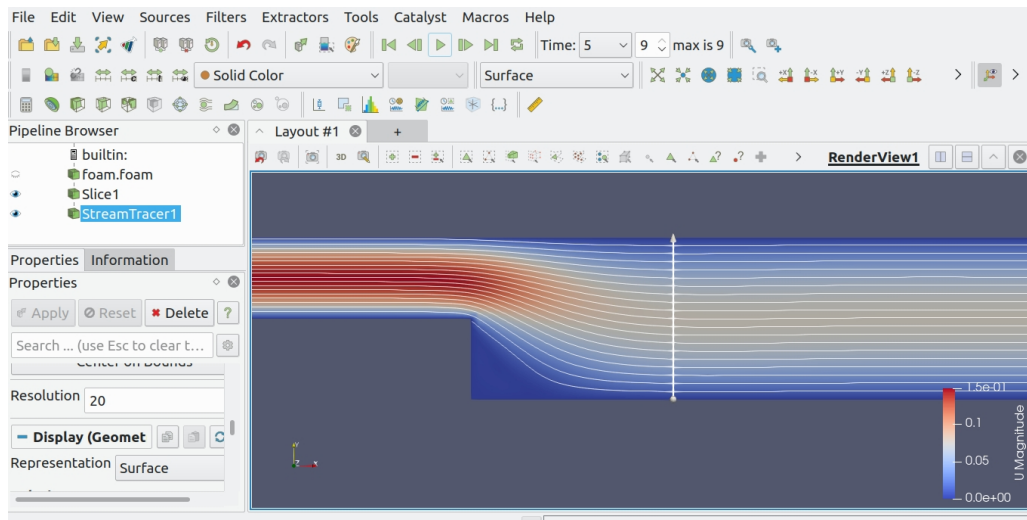


Agora você pode visualizar o plano central do escoamento. Mude para U a propriedade que está sendo visualizada e aperte o *play*. Visualize também as componentes de $\text{grad}(U)$.

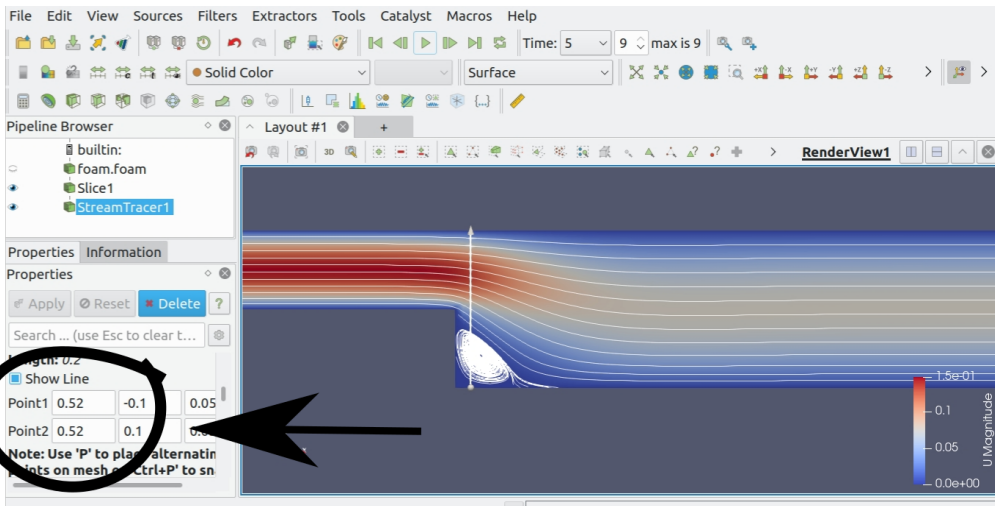


Agora vamos visualizar as linhas de corrente.

- 1 Clique em *Stream Tracer*, que está próximo da opção *Slice* (quatro casas para a direita).
- 2 Em *Properties*, no canto esquerdo central da tela, deslize as opções para baixo e clique em *Y Axis*
- 3 Em *Resolution*, substitua 1000 por 20.
- 4 Clique em *Apply*
- 5 Em *properties*, na opção *Coloring*, troque p por *Solid Color*.
- 6 Ative o olho da opção *Slice1* no *Pipeline Browser*.



Para visualizar a recirculação, posicione a linha do *Stream Tracer* em $x = 0.52$.



Um dos resultados mais importantes neste problema, que pode ser comparado com resultados experimentais, é o tamanho da recirculação principal.

Para podermos definir o tamanho dessa recirculação de forma precisa, vamos usar o gradiente da velocidade.

Vamos plotar o valor de $\frac{\partial U_1}{\partial x_2}$ na parede de baixo, na região da saída.

O ponto em que $\frac{\partial U_1}{\partial x_2}$ passa de negativo para positivo indica o fim da recirculação.

O $\text{grad}(\mathbf{U})$ possui 9 componentes:

$$\text{grad}(\mathbf{U}) = \nabla \mathbf{U} = \begin{bmatrix} \frac{\partial U_1}{\partial x_1} & \frac{\partial U_2}{\partial x_1} & \frac{\partial U_3}{\partial x_1} \\ \frac{\partial U_1}{\partial x_2} & \frac{\partial U_2}{\partial x_2} & \frac{\partial U_3}{\partial x_2} \\ \frac{\partial U_1}{\partial x_3} & \frac{\partial U_2}{\partial x_3} & \frac{\partial U_3}{\partial x_3} \end{bmatrix}$$

Como a contagem começa a partir do zero, nós queremos a componente 3 de $\text{grad}(\mathbf{U})$.

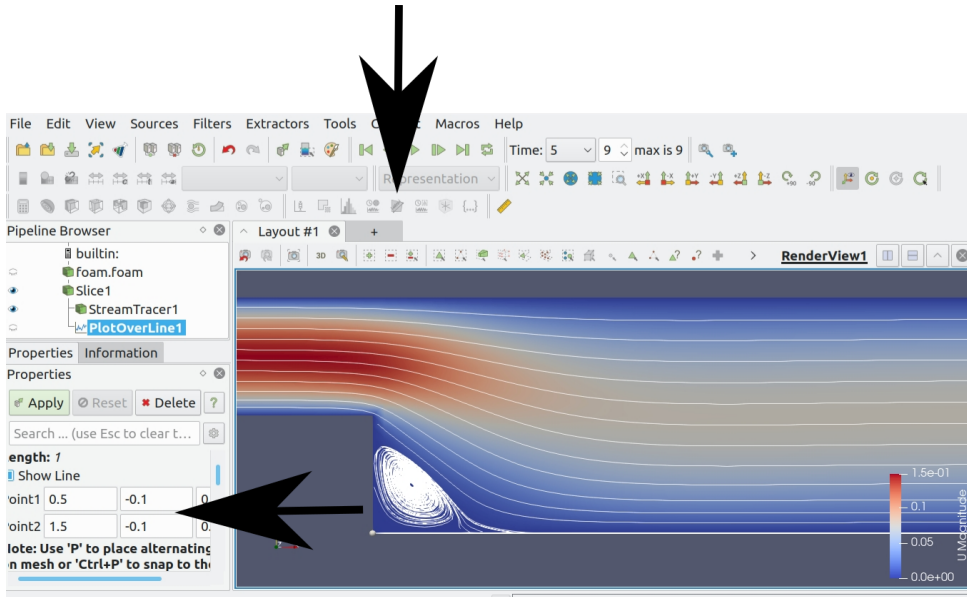
Com *Slice1* selecionado no *Pipeline Browser*,

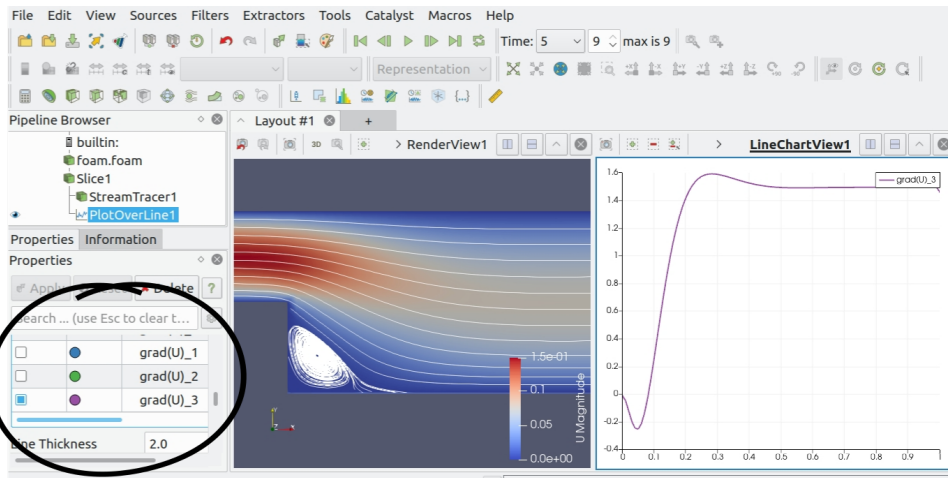
1 clique em *Plot Over Line*

2 em propriedades, defina o ponto 1 como sendo 0.5, -0.1, 0.05 e o ponto 2 como sendo 1.5, -0.1, 0.05.

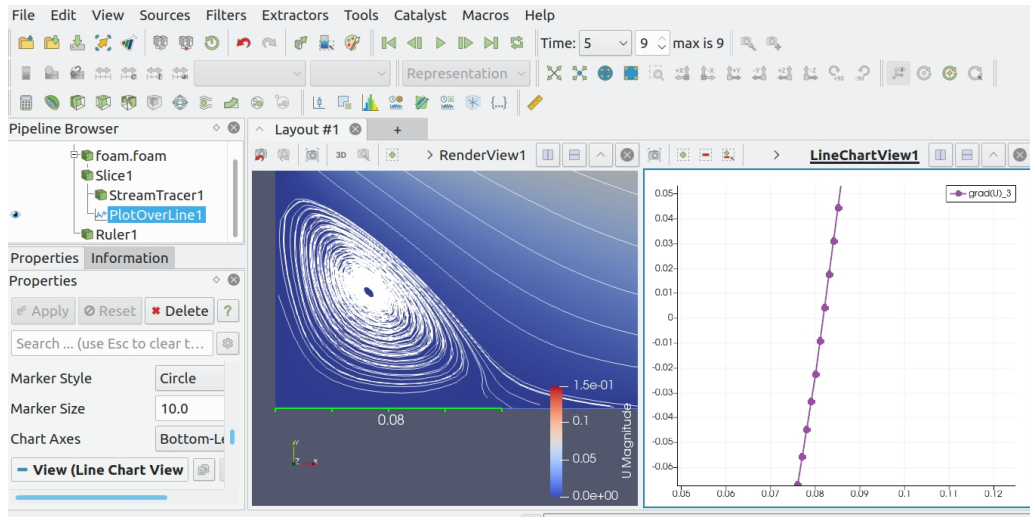
3 clique em *Apply*

4 em *Properties*, deixe apenas a opção $\text{grad}(U)$ selecionada na tabela *Series Parameters*.





Resultado final. Exporte suas figuras em File -> Save Screenshot .



Pronto!! Esse é o escoamento em degrau.

Rode outras simulações com diferentes números de Reynolds e diferentes tamanhos de entrada e de saída.

Plote o vetor velocidade. Veja como a simulação avança no tempo.

Compare os seus resultados com resultados da literatura.

Essa simulação pode ser usada como base para a simulação da junção T.

Sumário

- 1 Introdução
- 2 Escoamento em Degrau
- 3 OpenFOAM - Pastas**
- 4 OpenFOAM - Tutoriais
 - damBreak
 - damBreakFine em Paralelo
 - pitzDailySteady
 - airFoil2D
 - coolingCylinder2D
- 5 OpenFOAM - Dicas

Entre na pasta onde estão os arquivos e códigos do OpenFOAM:

```
/opt/openfoam11
```

Nesta pasta você vai encontrar 9 pastas:

- 1) *applications*: esta pasta contém os aplicativos (*solvers*, *modules* e *utilities*) CFD específicos que podem ser usados para realizar simulações em uma ampla variedade de problemas.

- 2) *bin*: aqui estão os executáveis (programas) compilados do OpenFOAM, que podem ser chamados a partir da linha de comando para executar tarefas específicas, como configuração de simulações ou processamento de dados.
- 3) *doc*: esta pasta contém documentação, manuais e tutoriais que auxiliam os usuários na compreensão e utilização do OpenFOAM. Isso é valioso para aprender a usar o software de maneira eficaz. O *User Guide* está nesta pasta.

- 4) *etc*: a pasta *etc* contém arquivos de configuração e *scripts* que são usados para personalizar o ambiente do OpenFOAM, como configurações de inicialização e parâmetros padrão.
- 5) *platforms*: esta pasta é usada para armazenar informações sobre diferentes plataformas (sistemas operacionais e arquiteturas) suportadas pelo OpenFOAM. Pode conter *scripts* e configurações específicas de plataforma.

- 6) *src*: pasta com o código-fonte do OpenFOAM, ou seja, os arquivos de código-fonte que são usados para compilar o *software*. Desenvolvedores e usuários avançados acessam esta pasta para personalizar ou estender o OpenFOAM.
- 7) *test*: aqui estão localizados os casos de teste e validação que garantem que o OpenFOAM funcione corretamente em diferentes cenários. Isso é importante para verificar a precisão das simulações.

- 8) *tutorials*: contém uma série de exemplos prontos para uso que demonstram como configurar e executar simulações para casos comuns. É uma ótima maneira de aprender a usar o OpenFOAM. Você pode usar um tutorial como base para desenvolver o seu caso de estudo.
- 9) *wmake*: esta pasta contém *scripts* e arquivos relacionados à construção (compilação) do OpenFOAM. É onde a compilação do código-fonte é gerenciada.

No OpenFOAM nós temos as chamadas *applications*, que podem ser *solvers*, *modules* ou *utilities*.

Exemplos de *solvers*: *foamRun* e *foamMultiRun*.

Exemplos de *modules*: *incompressibleFluid*, *incompressibleVoF* e *shock-Fluid*.

Exemplos de *utilities*: *blockMesh*, *foamPostProcess* e *setFields*.

Comentário: a versão 11 do OpenFOAM apresenta uma mudança significativa com relação às versões anteriores, que é a inclusão dos *modules*.

Nas versões anteriores nós tínhamos vários *solvers*. Agora, na versão 11, nós temos poucos *solvers* e vários *modules*.

Essa mudança deixa o código mais organizado e ajuda na simulação de problemas com vários domínios conectados, como no caso de Conjugate Heat Transfer.

O OpenFOAM foi criado na década de 1980 por Henry Weller e Hrvoje Jasak.

O OpenFOAM é desenvolvido na linguagem de programação C++ e usa programação orientada a objeto.

A utilização de *templates* e herança reduz a duplicação de código e cria hierarquias de classes que impõem uma estrutura geral ao código.

A equação

$$\frac{\partial \rho \mathbf{U}}{\partial t} + \nabla \cdot (\phi \mathbf{U}) - \nabla \cdot (\mu \nabla \mathbf{U}) = -\nabla p$$

pode ser escrita no OpenFOAM como

```
1 solve
2 (
3   fvm::ddt(rho, U)
4   + fvm::div(phi, U)
5   - fvm::laplacian(mu, U)
6   ==
7   - fvc::grad(p)
8 );
```

Uma outra forma de escrever essa equação, com mais opções e deixando o problema mais geral, é apresentada abaixo (do arquivo *modules/isothermalFluid/momentumPredictor.C*).

```
1 void Foam::solvers::isothermalFluid::momentumPredictor()
2 {
3     volVectorField& U(U_);
4
5     tUEqn =
6     (
7         fvm::ddt(rho, U) + fvm::div(phi, U)
8         + MRF.DDt(rho, U)
9         + momentumTransport->divDevTau(U)
10        ==
11        fvModels().source(rho, U)
12    );
13     fvVectorMatrix& UEqn = tUEqn.ref();
14
15     UEqn.relax();
16
17     fvConstraints().constrain(UEqn);
```

```
18  if (pimple.momentumPredictor())
19  {
20      if (buoyancy.valid())
21      {
22          solve
23          (
24              UEqn
25              ==
26              netForce()
27          );
28      }
29      else
30      {
31          solve(UEqn == -fvc::grad(p));
32      }
33
34      fvConstraints().constrain(U);
35      K = 0.5*magSqr(U);
36  }
37 }
```


Não vamos entrar no código fonte do OpenFOAM agora. Mas é bom dar uma olhada para ir conhecendo alguns detalhes.

Caso você tenha curiosidade, explore o site

<https://cpp.openfoam.org/v11/>

Lá você vai encontrar todo o código do OpenFOAM e vai poder navegar pelos diferentes arquivos e classes de forma interativa.

Agora nós vamos rodar alguns tutoriais.

Sumário

- 1 Introdução
- 2 Escoamento em Degrau
- 3 OpenFOAM - Pastas
- 4 OpenFOAM - Tutoriais**
 - damBreak
 - damBreakFine em Paralelo
 - pitzDailySteady
 - airFoil2D
 - coolingCylinder2D
- 5 OpenFOAM - Dicas

Entre na pasta

```
/opt/openfoam11/tutorials
```

Esta pasta contém mais de 100 casos (tutoriais) prontos para rodar, envolvendo os mais diversos tipos de problemas em CFD.

Esta pasta é importantíssima para o usuário do OpenFOAM, pois você sempre começa um estudo de caso copiando um tutorial que seja parecido com o problema que você está estudando.

Além disso, é possível aprender muito sobre como configurar as simulações a partir dos casos dos tutoriais.

Reserve um tempo para explorar os tutoriais do OpenFOAM com calma. Rode diferentes casos e veja a solução no Paraview. Procure tutoriais que têm características similares às do problema que você está investigando.

Vamos ver alguns tutoriais aqui.

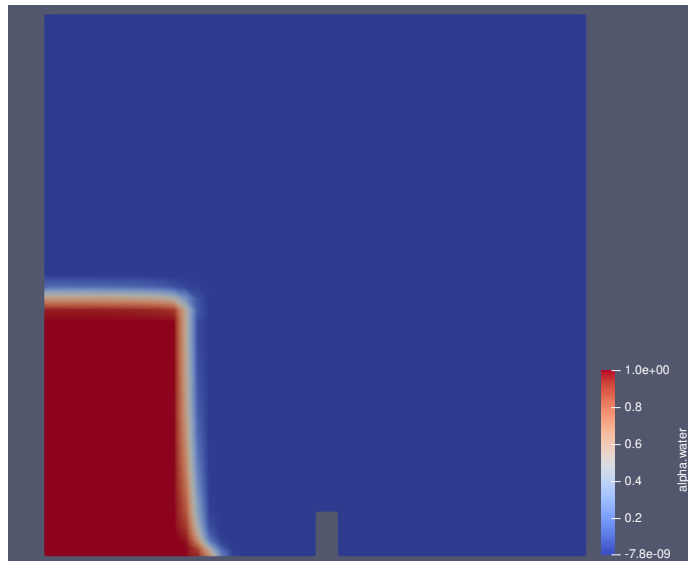
Sumário

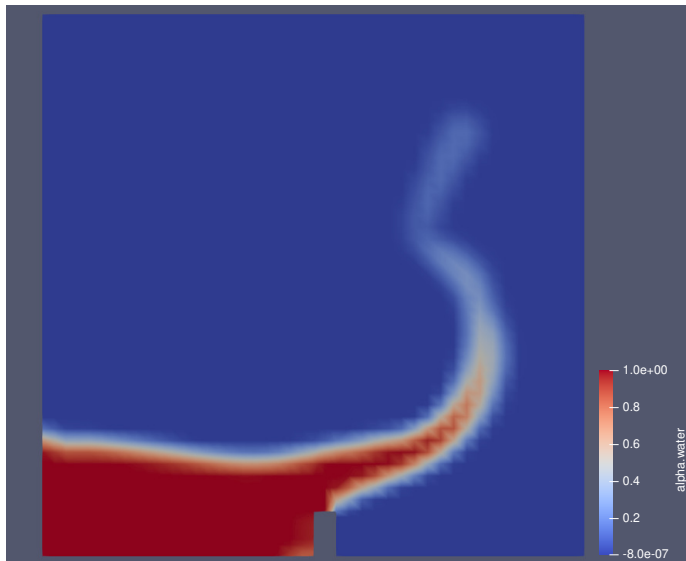
- 1 Introdução
- 2 Escoamento em Degrau
- 3 OpenFOAM - Pastas
- 4 OpenFOAM - Tutoriais
 - **damBreak**
 - damBreakFine em Paralelo
 - pitzDailySteady
 - airFoil2D
 - coolingCylinder2D
- 5 OpenFOAM - Dicas

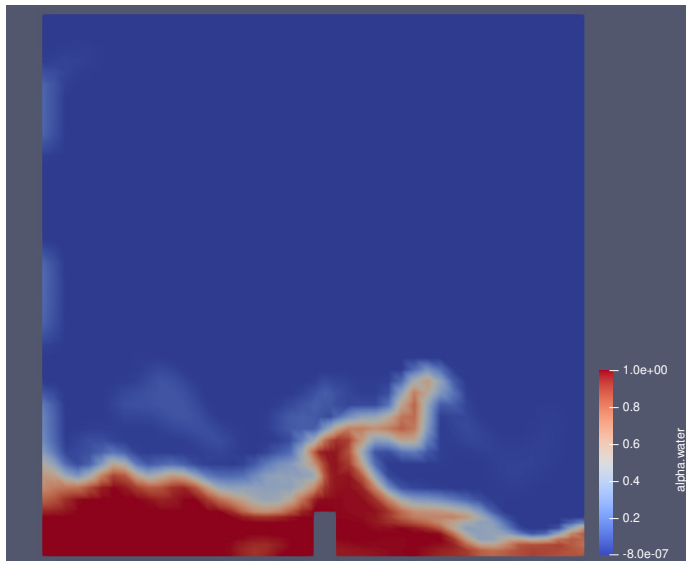
Tutorial damBreak.

Escoamento bifásico de água e ar.

Simula uma barragem se rompendo.







Copie a pasta

`/opt/openfoam11/tutorials/incompressibleVoF/damBreak/damBreak`

para uma pasta de seu computador.

Atenção: copie a pasta damBreak que está dentro da pasta damBreak.

Atenção: você deve copiar para uma pasta que você possa editar. (No Windows, pode ser em Meus Documentos ou nas suas pastas pessoais mesmo.)

Na pasta copiada, você vai encontrar 3 pastas e dois arquivos *scripts* executáveis.

As pastas são as tradicionais *0*, *constant* e *system*.

Os dois executáveis são o *Allrun* e o *Allclean*. Ambos são *scripts* que vão executar comandos do OpenFOAM na pasta.

O *Allrun* vai rodar o caso e o *Allclean* vai limpar a solução e deixar o caso como estava originalmente.

Pra rodar o caso, basta executar o Allrun. Execute o comando:

```
$: ./Allrun
```

Quando você executa esse comando, na verdade você está executando os comandos *blockMesh*, *setFields* e *foamRun*, nesta ordem. (Você pode tentar executar os 3 comandos individualmente, pra ver que o resultado é o mesmo.)

Esse tutorial simula um escoamento bifásico: temos água e ar. A ideia é simular uma barragem se rompendo.

Uma das ferramentas que o OpenFOAM tem para simular escoamentos multifásicos é a metodologia de Volume de Fluido (*Volume of Fluid*, ou *VoF*).

O comando *blockMesh* nós já conhecemos: ele vai gerar a malha a partir do arquivo *system/blockMeshDict* .

Aproveite para dar uma olhada neste arquivo.

Depois nós temos o comando *setFields* .

Esse comando gera a distribuição inicial de água e ar no domínio, de acordo com o arquivo *system/setFieldsDict* .

Vamos dar uma olhada no arquivo.

system/setFieldsDict

```
1 defaultFieldValues
2 (
3     volScalarFieldValue alpha.water 0
4 );
5
6 regions
7 (
8     boxToCell
9     {
10         box (0 0 -1) (0.1461 0.292 1);
11         fieldValues
12         (
13             volScalarFieldValue alpha.water 1
14         );
15     }
16 );
```

Neste caso, nós temos duas fases. A variável *alpha* representa a fração volumétrica de cada fase.

alpha.water representa a fração volumétrica de água e *alpha.air* representa a fração volumétrica de ar.

Nos pontos do domínio em que *alpha.water* é zero, nós temos ar, e nos pontos em que *alpha.water* = 1 nós temos água.

A soma das duas frações deve ser 1, ou seja,

$$\alpha_{water} + \alpha_{air} = 1$$

Assim, se o *alpha* de uma fase é conhecido, o outro *alpha* já é obtido automaticamente.

No arquivo *setFieldsDict*, primeiro estamos definindo o valor *default* de *alpha.water* como sendo zero em todo o domínio.

Depois, estamos definindo regiões onde *alpha.water* é igual a 1.

No caso, *alpha.water* vai ser igual a 1 em um *box* limitado pelos vértices $(0 \ 0 \ -1)$ e $(0.1461 \ 0.292 \ 1)$.

Na pasta *constant*, nós temos um arquivo com as propriedades do ar (*physicalProperties.air*) e um arquivo com as propriedades da água (*physicalProperties.water*), além de um arquivo com o valor da tensão superficial entre as fases (*phaseProperties*).

Na pasta *constant* nós também temos um arquivo com o modelo de turbulência utilizado (*momentumTransport*) e um arquivo indicando a intensidade e a direção do vetor aceleração gravitacional (*g*).

Por fim, no arquivo *fvModels* nós temos a definição do parâmetro *turbulenceDamping*, que é um parâmetro que pode ser usado VOF para controlar o amortecimento da turbulência nas interfaces entre as fases.

Na pasta 0 nós temos as condições iniciais e de contorno para cada variável.

As variáveis *epsilon*, *k*, *nut*, *nuTilda* e *omega* estão relacionadas ao modelo de turbulência. Iremos estudá-las mais adiante.

O arquivo *alpha.water.orig* apresenta as condições para a fração volumétrica de água.

U é a velocidade e p_rgh é a pressão reduzida, que é a pressão menos ρgh .

A parede de cima está aberta para a atmosfera e permite a entrada e a saída de fluido. Dê uma olhada nos arquivos *0/U* e *0/p_rgh* pra ver como essa condição de contorno é implementada.

Depois de executar *Allrun*, abra a solução no *Paraview* e explore bem os resultados.

Visualize a variável *alpha.water* .

Para limpar o caso e deixá-lo como estava no tutorial, execute o comando

```
$: ./Allclean
```

Pronto! Rodamos o tutorial *damBreak*.

Explore mais um pouco esse tutorial. Mude a viscosidade da água (coloque uma viscosidade bem maior), mude a posição inicial da água, mude a tensão superficial, Use a sua criatividade!

Sumário

- 1 Introdução
- 2 Escoamento em Degrau
- 3 OpenFOAM - Pastas
- 4 OpenFOAM - Tutoriais
 - damBreak
 - **damBreakFine em Paralelo**
 - pitzDailySteady
 - airFoil2D
 - coolingCylinder2D
- 5 OpenFOAM - Dicas

Vamos rodar de novo o *damBreak*, mas agora com uma malha mais refinada e em paralelo.

Vamos aproveitar o caso *damBreak* e vamos modificar o arquivo *blockMeshDict*.

Vá para a pasta onde a sua pasta *damBreak* e execute o seguinte comando:

```
$: foamCloneCase damBreak/ damBreakFine
```

Uma pasta chamada *damBreakFine* vai ser criada. Nela estarão os arquivos de configuração do caso, idênticos aos da pasta *damBreak*.

Modifique a lista *blocks* no arquivo *system/blockMeshDict* para deixá-la da seguinte forma:

```
1 blocks
2 (
3     hex (0 1 5 4 12 13 17 16) (46 10 1) simpleGrading (1 1 1)
4     hex (2 3 7 6 14 15 19 18) (40 10 1) simpleGrading (1 1 1)
5     hex (4 5 9 8 16 17 21 20) (46 76 1) simpleGrading (1 2 1)
6     hex (5 6 10 9 17 18 22 21) (4 76 1) simpleGrading (1 2 1)
7     hex (6 7 11 10 18 19 23 22) (40 76 1) simpleGrading (1 2 1)
8 );
```


Em *system/controlDict*, ajuste o passo de tempo para 0.0005.

Pronto! Tudo ajustado.

Agora vamos rodar o caso. Mas desta vez vamos rodar em paralelo.

Isso significa que nós vamos rodar a simulação em mais de um processador. Quando a simulação é muito pesada, isso ajuda a acelerar os cálculos.

Primeiro, vamos precisar do arquivo *decomposeParDict*. Esse arquivo já está na pasta *system*. (Se não estivesse lá, você poderia usar o comando *foamGet decomposeParDict* .)

Neste caso, vamos dividir o domínio em 4 regiões e vamos rodar a simulação em 4 processadores simultaneamente. Isso está no arquivo *system/decomposeParDict* :

```
1 numberOfSubdomains 4;
2
3 method          simple;
4
5 simpleCoeffs
6 {
7     n            (2 2 1);
8 }
```

Para rodar o caso, execute os comandos:

```
$: blockMesh
```

```
$: setFields
```

```
$: decomposePar
```

```
$: mpirun -np 4 foamRun -parallel
```

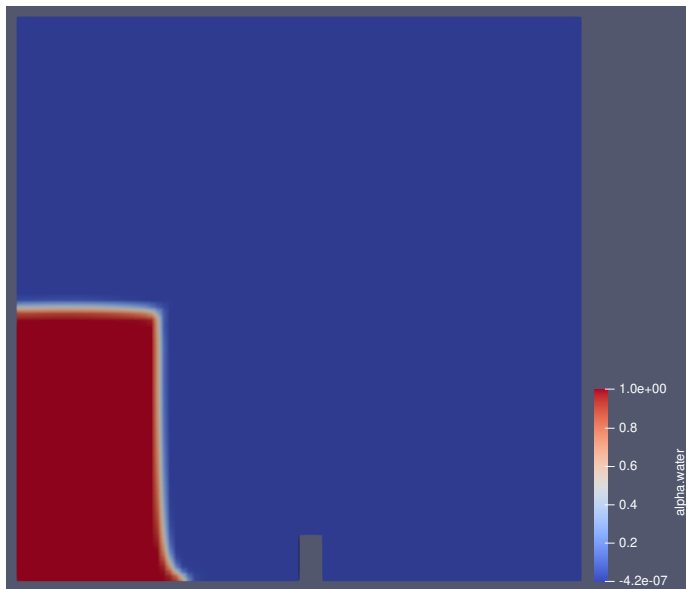
```
$: reconstructPar
```

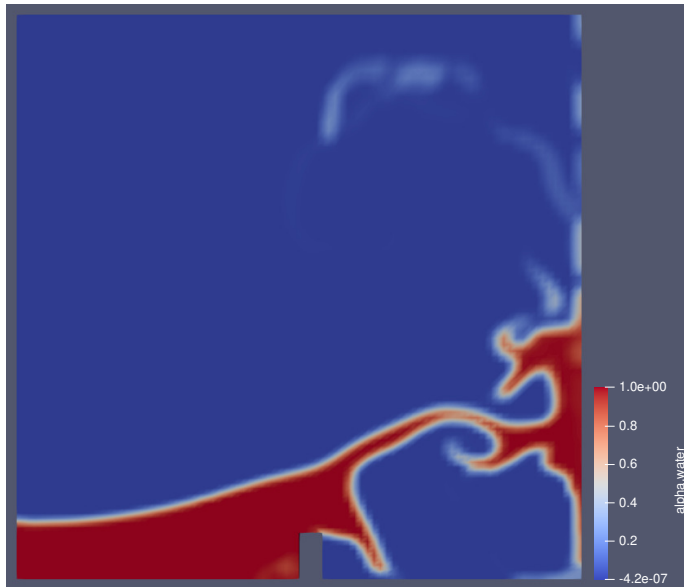
```
$: paraFoam
```

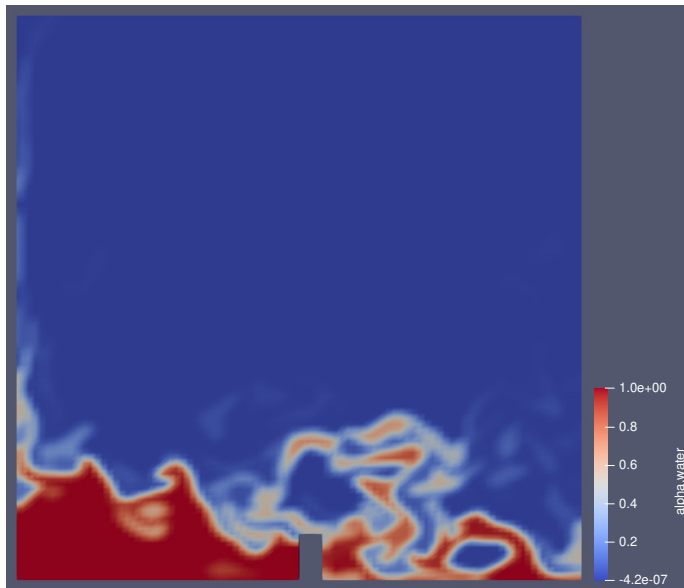
O comando *decomposePar* divide o domínio em 4 e o comando *mpirun* roda em paralelo.

Depois da simulação, o comando *reconstructPar* junta de novo as soluções dos 4 domínios, para podermos visualizar.

Os próximos slides mostram os novos resultados. Agora, com a malha melhor, podemos ver uma diminuição na interface entre a água e o ar, representando uma melhora na simulação do problema.







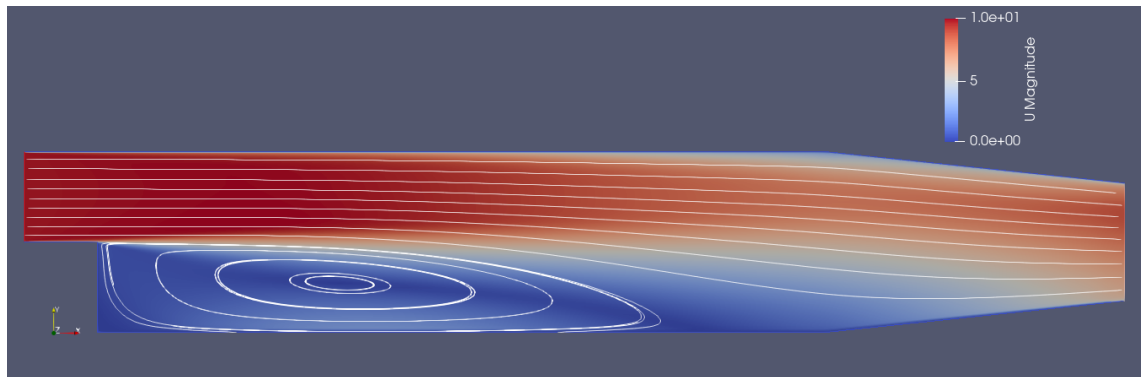
Sumário

- 1 Introdução
- 2 Escoamento em Degrau
- 3 OpenFOAM - Pastas
- 4 OpenFOAM - Tutoriais
 - damBreak
 - damBreakFine em Paralelo
 - **pitzDailySteady**
 - airFoil2D
 - coolingCylinder2D
- 5 OpenFOAM - Dicas

tutorials/incompressibleFluid/pitzDailySteady

O tutorial *pitzDailySteady* é projetado para estudar e simular o escoamento em um caso de validação chamado *PitzDaily*. O *PitzDaily* é um caso de referência comum usado para validar e verificar a precisão de solvers e configurações em simulações de dinâmica dos fluidos computacional (CFD). Especificamente, este tutorial se concentra em um cenário de escoamento em regime permanente (steady-state).

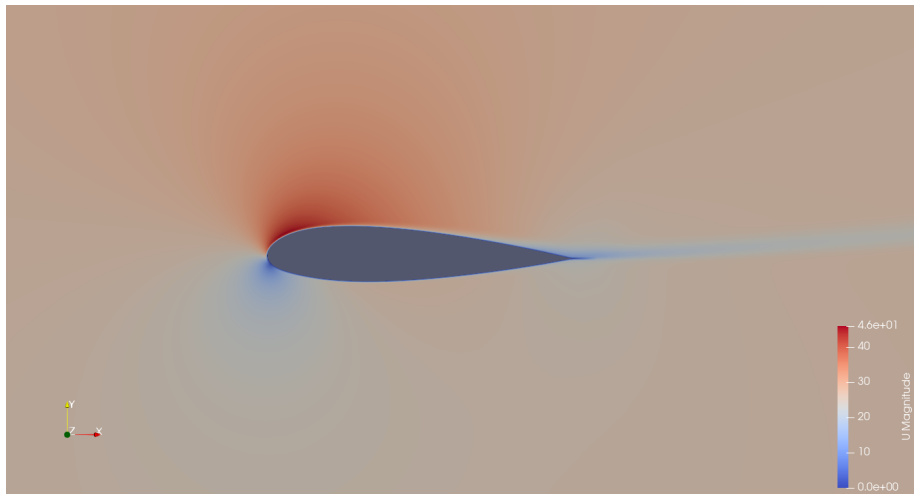
Tutorial interesssante para testar diferentes modelos de turbulência.

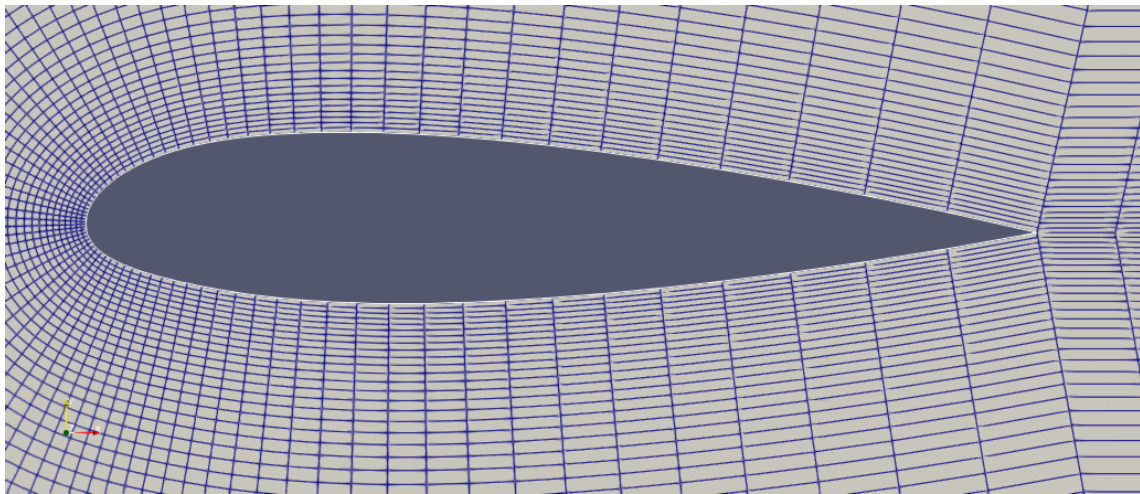


Sumário

- 1 Introdução
- 2 Escoamento em Degrau
- 3 OpenFOAM - Pastas
- 4 OpenFOAM - Tutoriais
 - damBreak
 - damBreakFine em Paralelo
 - pitzDailySteady
 - **airFoil2D**
 - coolingCylinder2D
- 5 OpenFOAM - Dicas

tutorials/incompressibleFluid/airFoil2D





Escoamento ao redor de um aerofólio.

Rode novas simulações com diferentes ângulos de ataque.

Visualize os campos de velocidade e de pressão.

Sumário

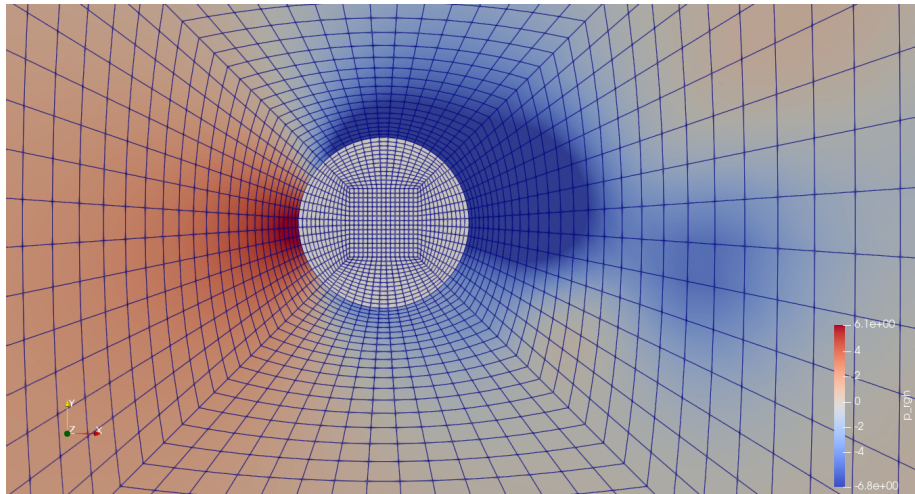
- 1 Introdução
- 2 Escoamento em Degrau
- 3 OpenFOAM - Pastas
- 4 OpenFOAM - Tutoriais
 - damBreak
 - damBreakFine em Paralelo
 - pitzDailySteady
 - airFoil2D
 - **coolingCylinder2D**
- 5 OpenFOAM - Dicas

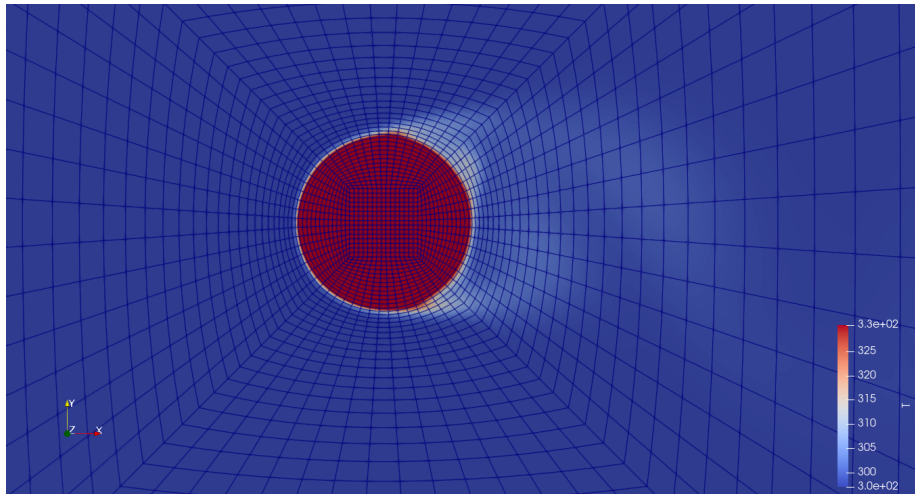
tutorials/multiRegion/CHT/coolingCylinder2D

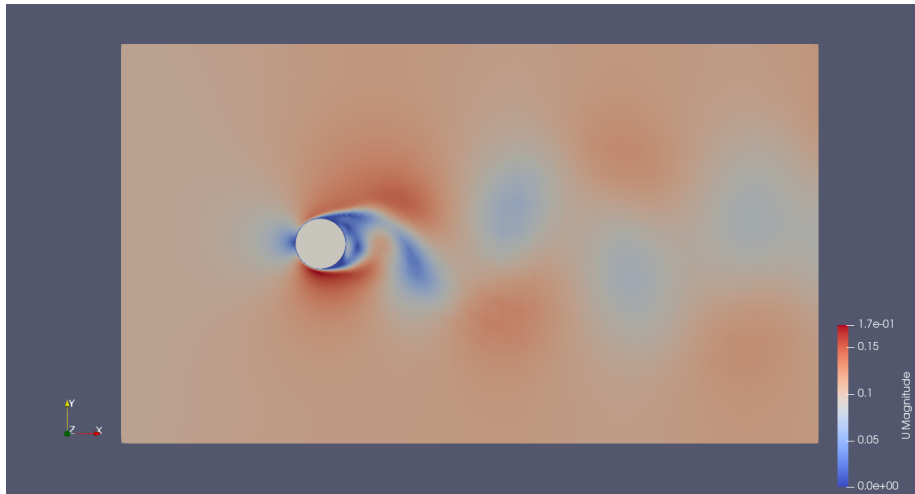
Tutorial de *Conjugate Heat Transfer*. Escoamento ao redor de um cilindro que está gerando calor. Neste caso estamos resolvendo o escoamento do fluido e também estamos resolvendo a equação da energia para o sólido.

Os dois domínios, do fluido e do sólido, estão conectados pelas condições de contorno da temperatura T na superfície do cilindro.

Neste tutorial é usado o solver *foamMultiRun* . Para o fluido é usado o módulo *fluid* e para o sólido é usado o módulo *solid*.







Sumário

- 1 Introdução
- 2 Escoamento em Degrau
- 3 OpenFOAM - Pastas
- 4 OpenFOAM - Tutoriais
 - damBreak
 - damBreakFine em Paralelo
 - pitzDailySteady
 - airFoil2D
 - coolingCylinder2D
- 5 OpenFOAM - Dicas

Nesta seção o meu objetivo é apresentar algumas dicas que podem ser úteis para você no uso do OpenFOAM.

foamInfo

Você pode obter informações sobre quase tudo no OpenFOAM com a *utility* *foamInfo*.

Por exemplo, execute

```
$: foamInfo incompressibleFluid
```

```
$: foamInfo fluid
```

```
$: foamInfo flowRateInlet
```

```
$: foamInfo newtonian
```

A saída do *foamInfo* explica o que é o objeto que você está pesquisando, mostra como usar e ainda indica tutoriais que usam esse objeto.

foamToC

Essa *utility* lista todas as opções que o usuário tem no OpenFOAM. ToC significa *Table of Content*.

Execute os comandos:

```
$: foamToC
```

```
$: foamToC -table solver
```

```
$: foamToC -table generalisedNewtonianViscosityModel
```


inlet boundary condition

Uma condição mais interessante de entrada de fluido é a *flowRateInletVelocity*.

Use o *foamInfo* para saber mais sobre ela.

Abaixo está um exemplo de seu uso.

```
1 inlet
2 {
3     type          flowRateInletVelocity;
4     meanVelocity   10;
5     profile        turbulentBL; // laminarBL;
6     value          uniform (10 0 0);
7 }
```

Pós processamento

O OpenFOAM possui diversas funções de pós processamento. Essas funções são usadas depois que a simulação já rodou. Com elas você pode extrair diversas informações do caso.

Execute o comando abaixo em qualquer pasta:

```
$: foamPostProcess -list
```

Em uma pasta de simulação, já com o resultado, execute o comando abaixo:

```
$: foamPostProcess -func "mag(U)"
```

banana

Se você quer saber todas as opções possíveis para uma variável, use o *banana trick*.

Coloque *banana* como valor para a variável e rode a simulação. Vai dar erro. Na mensagem de erro o OpenFOAM vai te falar que a opção *banana* não existe e vai listar todas as opções possíveis.

Por exemplo, no arquivo *system/controlDict* coloque *banana* como valor para a variável *startFrom*.

```
1 startFrom    banana;
```

Rode a simulação e veja quais são as opções para *startFrom*.

Comandos para acessar pastas do OpenFOAM.

Você pode acessar pastas do OpenFOAM com comandos curtos. Teste os comandos abaixo:

- foam
- tut
- mod
- src
- sol
- app

Para visualizar o conteúdo de uma pasta, use o comando *ls*.

Por enquanto são essas as dicas. Vamos ver outras ao longo do curso.